

Last lecture

What is (Computer) Vision?

„Computer Vision is the branch of Computer Science whose goal is to **model the real world** or to **recognize objects** from digital images.

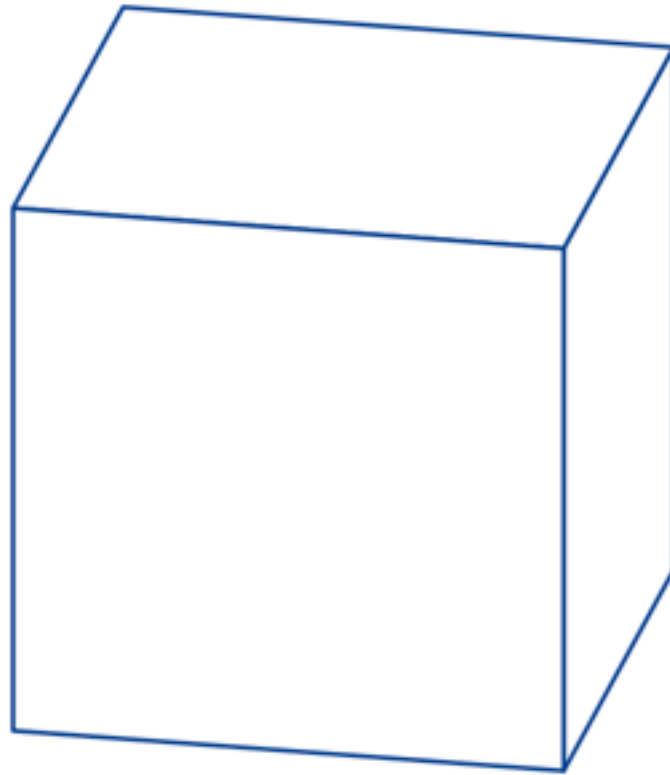
These images can be acquired using still and video cameras, infrared cameras, radars, or specialized sensors such as those used in the medical field.“

(Pascal Fua's Computer Vision Course @EPFL)

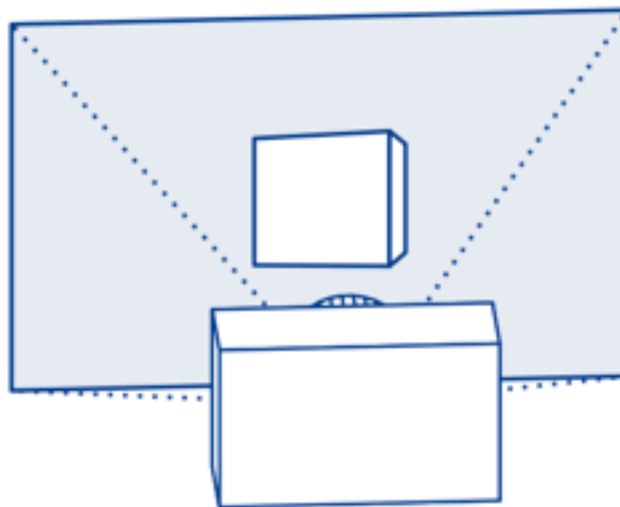
Vision versus Graphics

3D world

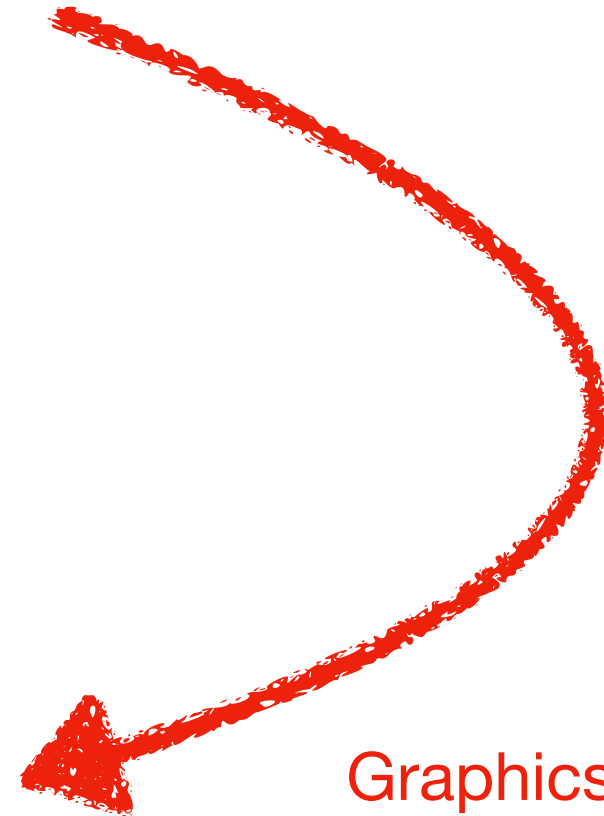
- Objects
- Shapes
- Materials
- Poses
- Motions
- Light



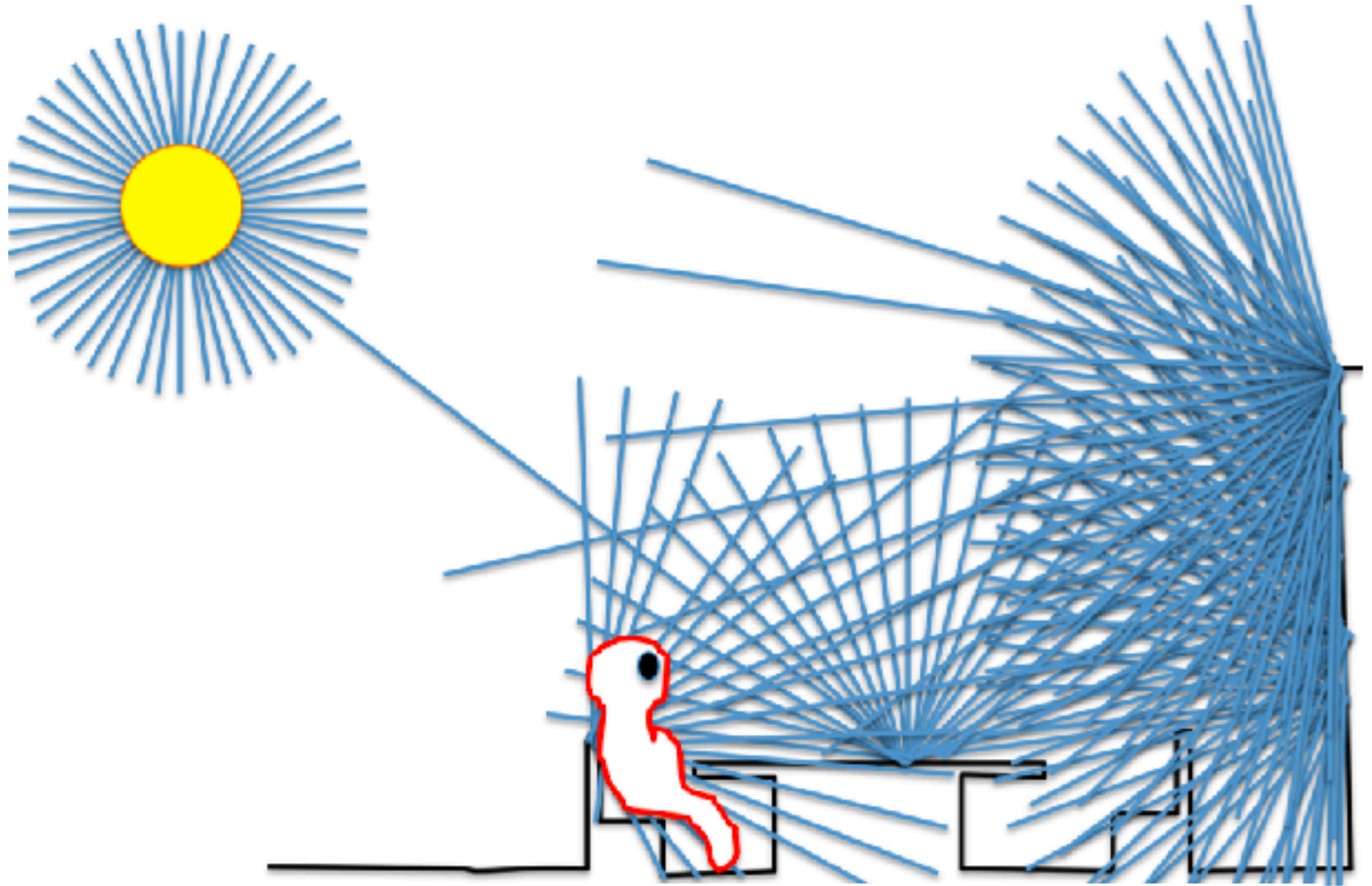
Vision:
Given images,
recover information
about the 3D scene



Graphics:
Given information of a
3D scene, produce
images



Turning light into meaning



The „plenoptic function“

The intensity P can be parameterized as:

$$P(\theta, \phi, \lambda, t, X, Y, Z)$$

“The complete set of all convergence points constitutes the permanent possibilities of vision.” Gibson

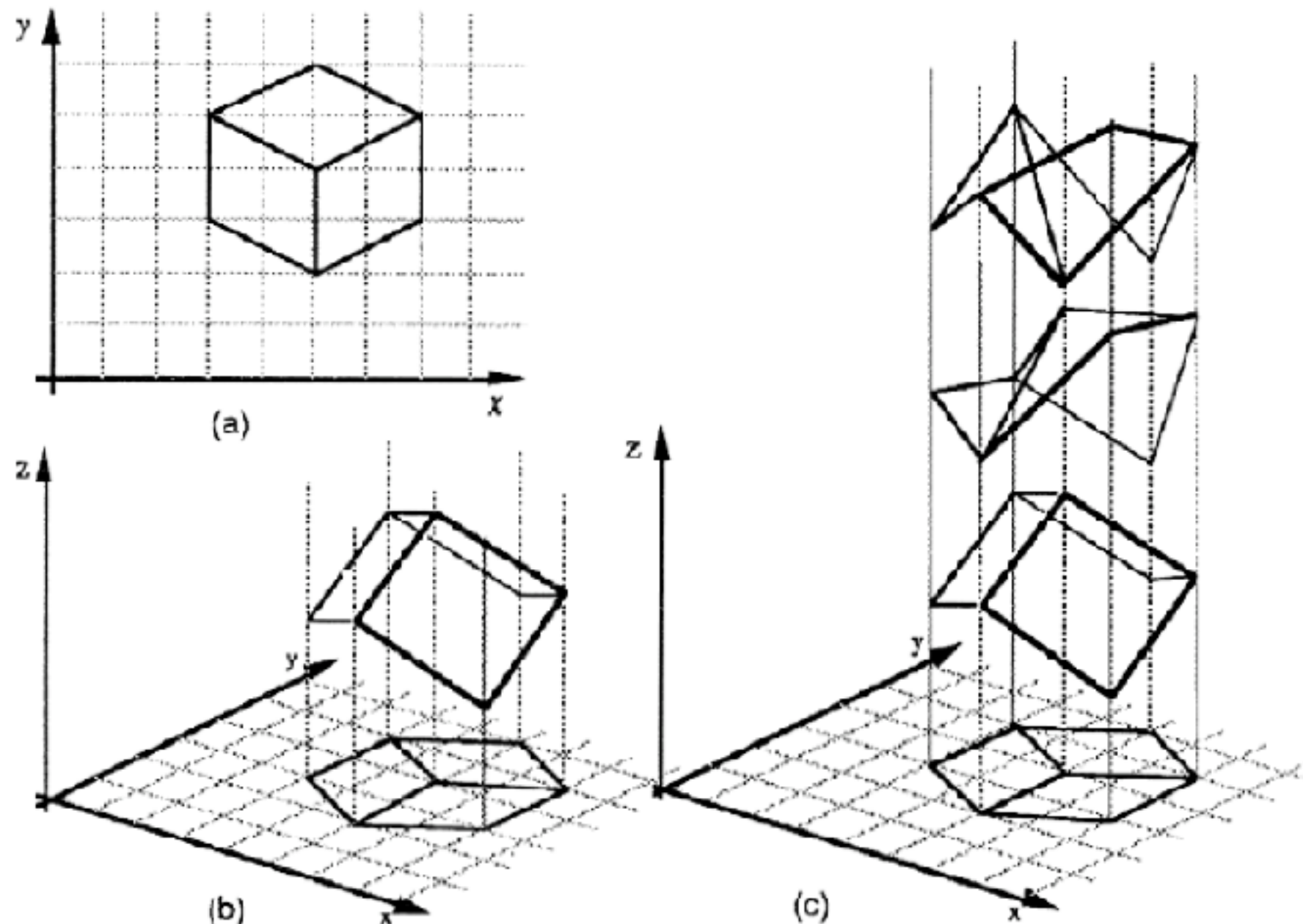
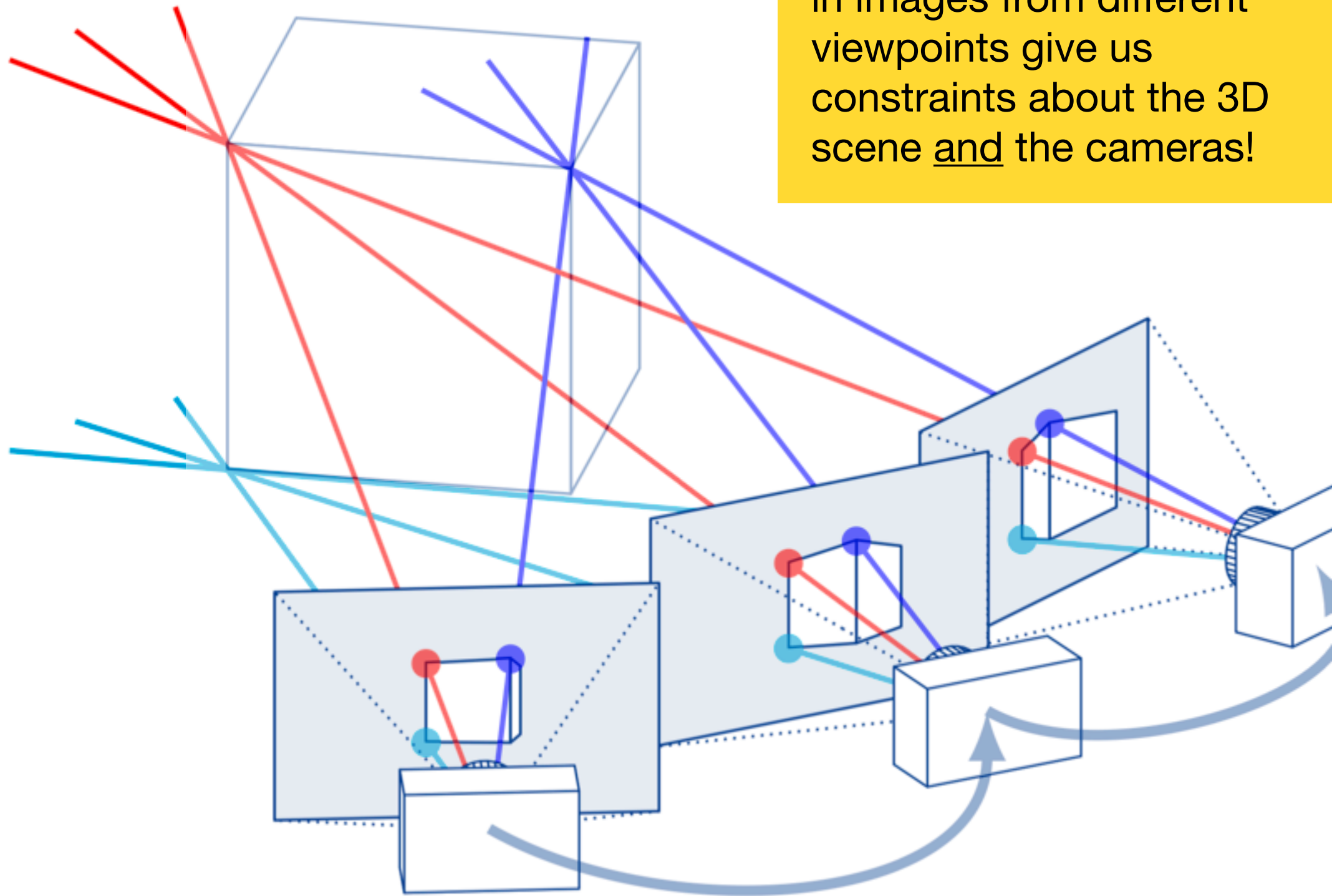


Figure 1. (a) A line drawing provides information only about the x, y coordinates of points lying along the object contours. (b) The human visual system is usually able to reconstruct an object in three dimensions given only a single 2D projection (c) Any planar line-drawing is geometrically consistent with infinitely many 3D structures.

Multiview geometry

Corresponding structures in images from different viewpoints give us constraints about the 3D scene and the cameras!



Computer vision is ill-posed!

- It is an inverse problem
- The projection is nonlinear and not invertible - information about the scene is lost during the projection
- The same image could result from many different scenes, and the same scene can result in many different images
- This effect can be nicely seen in visual illusion examples

Today

Today's goals

- Formalizing the concept of digital images and pixels
- Basics of sampling
- Understand (linear) 2D coordinate transformations

Notation

Unfortunately, math notation is often very different! In this lecture, we try to stick to the following conventions

- Scalar values are lowercase or uppercase letters such as x
- Vectors are bold lowercase and default to column order, such as

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \dots \\ v_n \end{bmatrix} = [v_1 \quad v_2 \quad \dots \quad v_n]^\top \in \mathbb{R}^n$$

- We sometimes write the short „tuple“ form $\mathbf{v} = (v_1, v_2, \dots, v_n)$ for vectors
- Matrices are bold uppercase, such as

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

The digital image

What is an image?

We consider an image as a mapping from a D -dimensional domain of coordinates to a C -dimensional domain of measured intensities:

$$f : \mathbb{R}^D \rightarrow \mathbb{R}^C$$

Pixels

A „pixel“ (picture element) is a point in an image, characterized by

- its position $\mathbf{x} = [x_1 \ \dots \ x_D]^\top \in \mathbb{R}^D$
- its value $\mathbf{v} = [v_1 \ \dots \ v_C]^\top \in \mathbb{R}^C$

It can thus be considered a tuple

$$p = (\mathbf{x}, \mathbf{v})$$

Discrete images

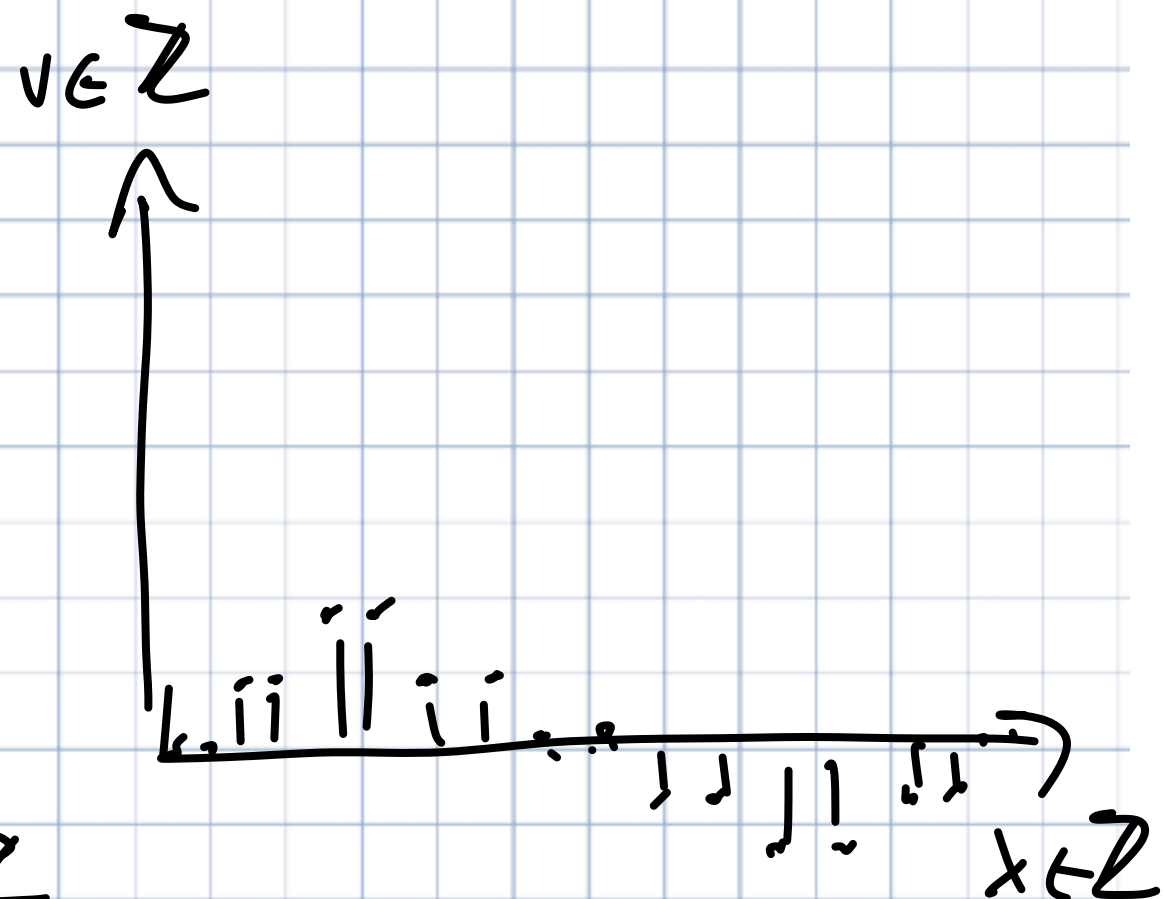
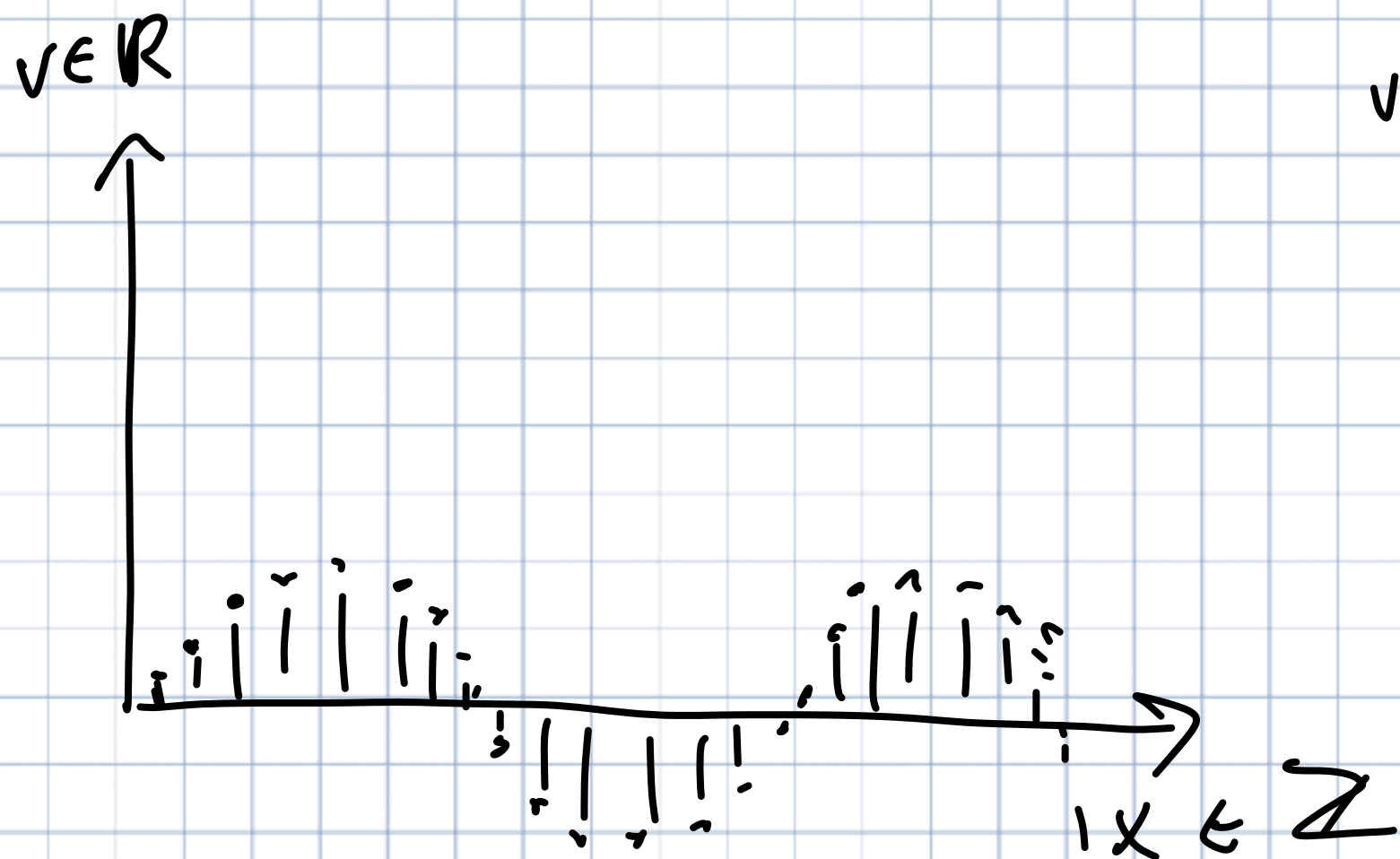
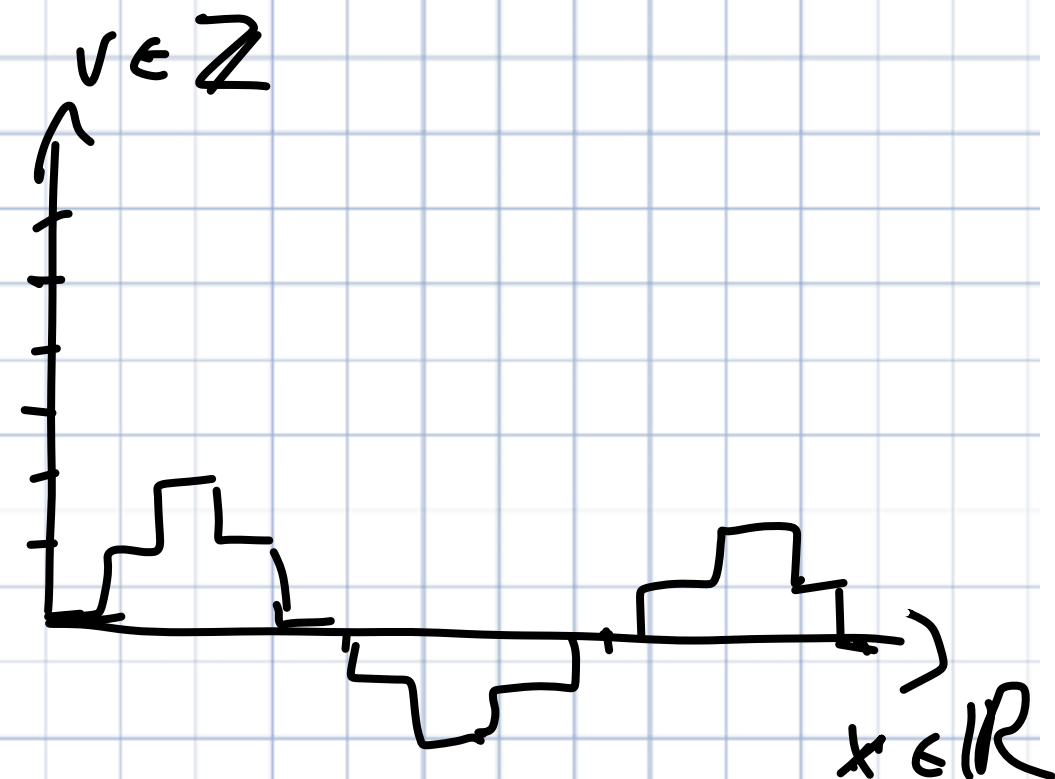
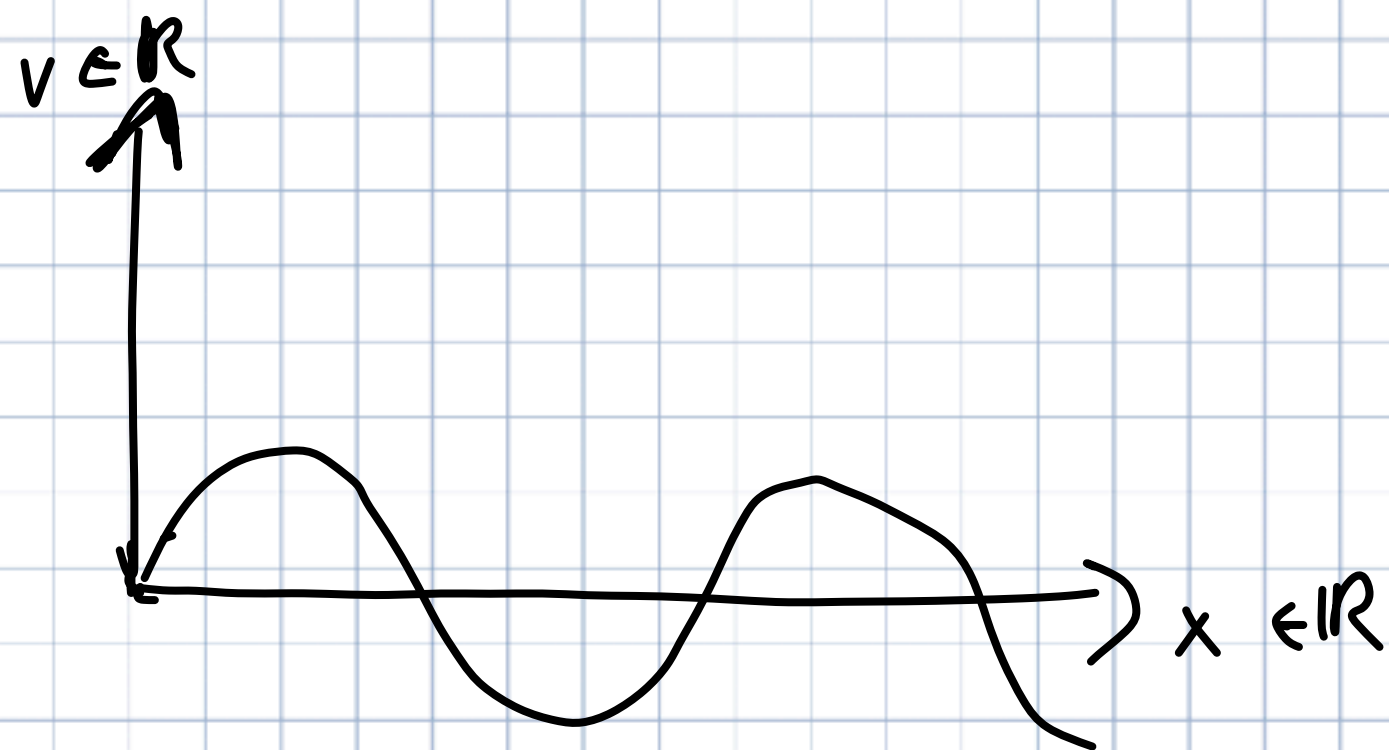
In digital images, we often have a discrete and finite 2D grid of coordinates ($D = 2$), sometimes even a discrete intensity domain.

Common case: 2-D image with 8-bit grayscale ($C = 1$):

$$f_{\text{gray}} : \mathbb{Z} \times \mathbb{Z} \rightarrow \{v \in \mathbb{Z} \mid 0 \leq v < 2^8\}$$

Note: Images with discrete coordinate grids and finite dimensions are n-dimensional arrays, and can be notated as matrices.

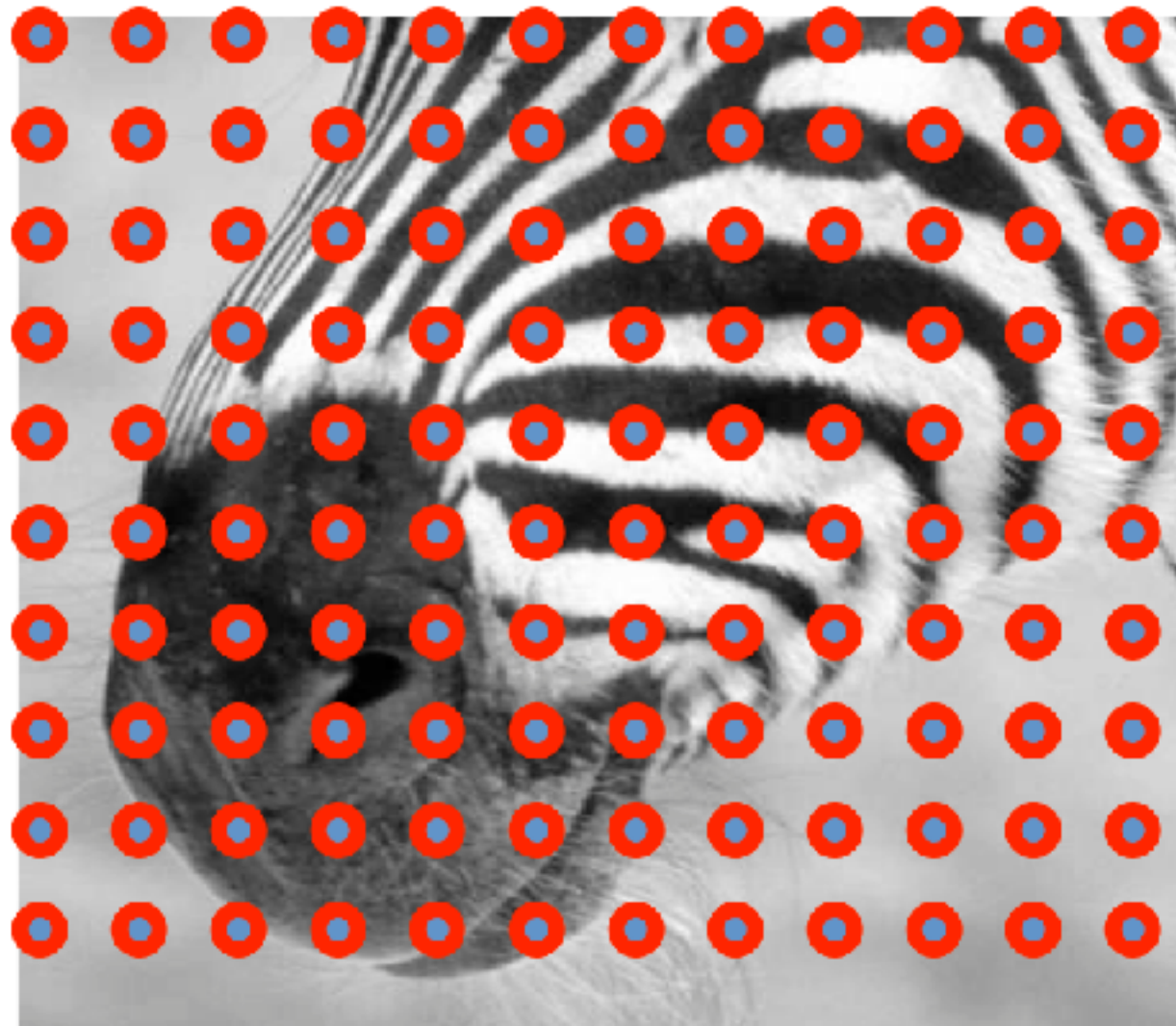
Quantization in the source and target domain



Quantization of images

- Coordinates: by the placement of photo diodes, e.g a 2048x1536 grid
- Intensities:
 - During A/D conversion (e.g. 16bit)
 - During signal processing (e.g. convert to 8-bit jpeg)

Subsampling by a factor of 2



Throw away every other row and column to create a 1/2 size image

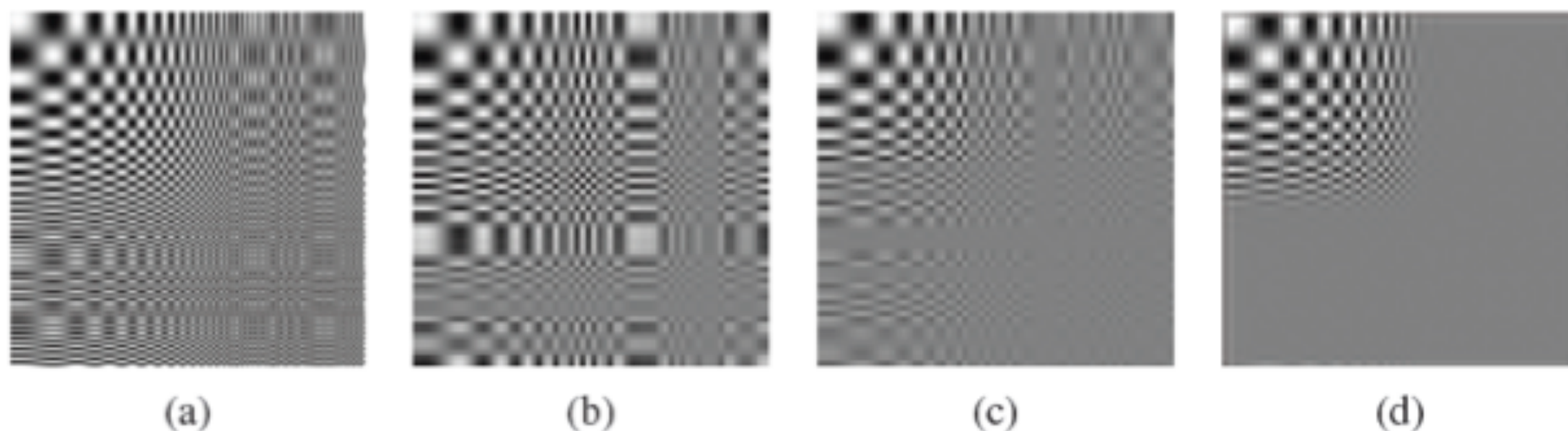


Figure 2.25 Aliasing of a two-dimensional signal: (a) original full-resolution image; (b) downsampled $4\times$ with a 25% fill factor box filter; (c) downsampled $4\times$ with a 100% fill factor box filter; (d) downsampled $4\times$ with a high-quality 9-tap filter. Notice how the higher frequencies are aliased into visible frequencies with the lower quality filters, while the 9-tap filter completely removes these higher frequencies.

(aus Szeliski 2010: "Computer Vision: Algorithms and Applications")



Let's code a bit!



Points and lines in 2D

2D points

The 2D point $\mathbf{x} = (x, y) = [x \ y]^T$ can be represented in *homogenous coordinates*

$$\tilde{\mathbf{x}} = (\tilde{x}, \tilde{y}, \tilde{w}) = \tilde{w}(x, y, 1) = \tilde{w}\bar{\mathbf{x}} \in \mathbb{P}^2$$

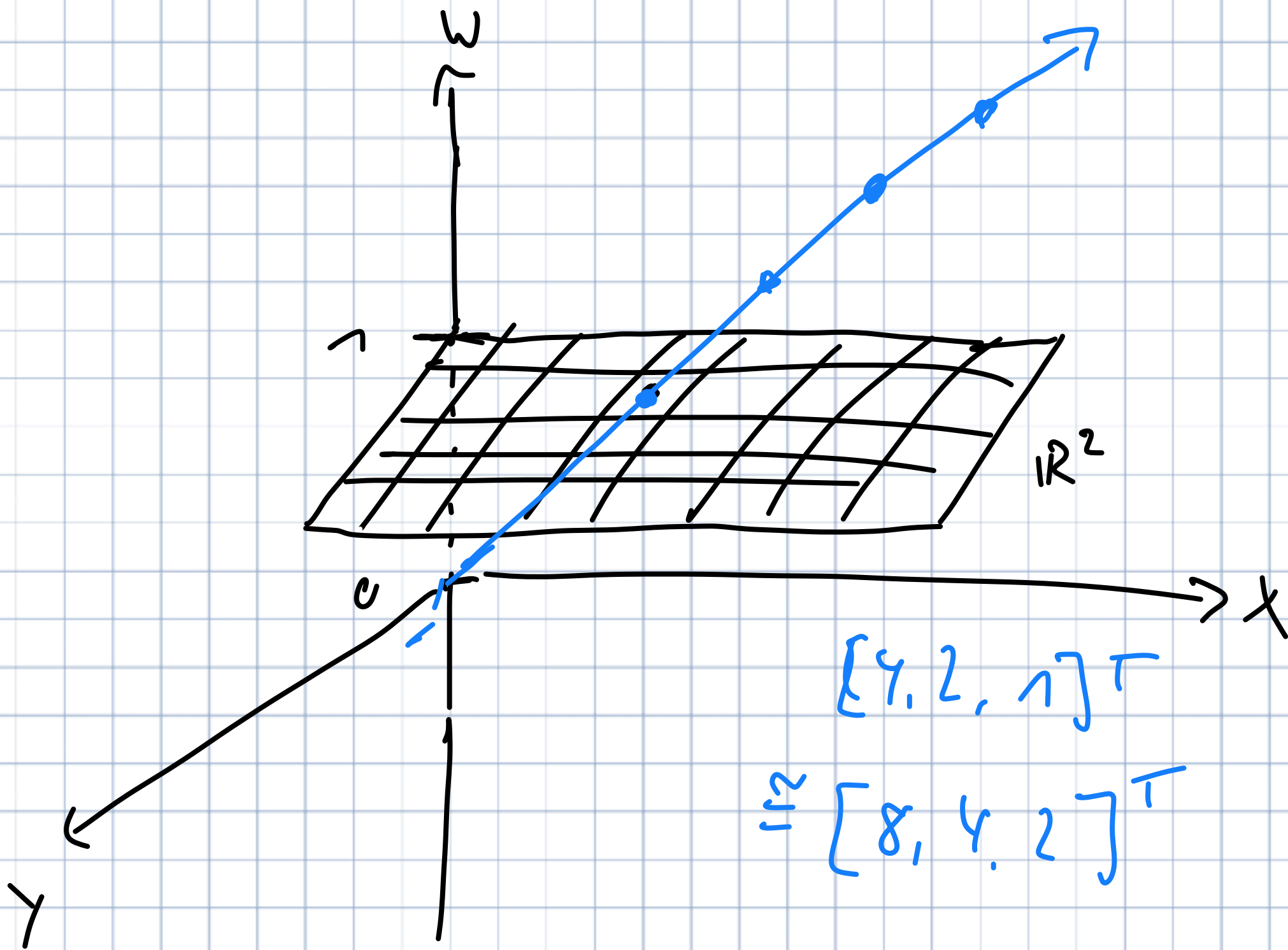
where

- scaled points are identical
- dividing by the 3rd coordinate gives the original Euclidean point
- dividing by the length gives us the point on the sphere
- What does $\tilde{w} = 0$ mean?

$$x = (2, 3) \rightarrow \tilde{x} = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} \approx \begin{bmatrix} 4 \\ 6 \\ 2 \end{bmatrix} \approx \begin{bmatrix} 8 \\ 12 \\ 4 \end{bmatrix}$$

$\downarrow :4$

normalization



**What's
wrong
here?**

It is a dark time for the Rebellion. Although the Death Star has been destroyed, Imperial troops have driven the Rebel forces from their hidden base and pursued them across the galaxy.

Evading the dreaded Imperial Starfleet, a group of freedom fighters led by Luke Skywalker has established a new secret base on the remote ice world of Hoth.

It is a dark time for the Rebellion. Although the Death Star has been destroyed, Imperial troops have driven the Rebel forces from their hidden base and pursued them across the galaxy.

Evading the dreaded Imperial Starfleet, a group of freedom fighters led by Luke Skywalker has established a new secret base on the remote ice world of Hoth.

2D geometric image transformations (Modifying the coordinates)



GIMP

GNU IMAGE
MANIPULATION PROGRAM

 **DOWNLOAD 2.10.34**

RELEASE NOTES

Image transformations

Transformations of 2D images may be defined as a function modifying the pixels. This may affect

- the pixel intensities or
- the pixel coordinates.

2D translation

The 2D coordinate translation

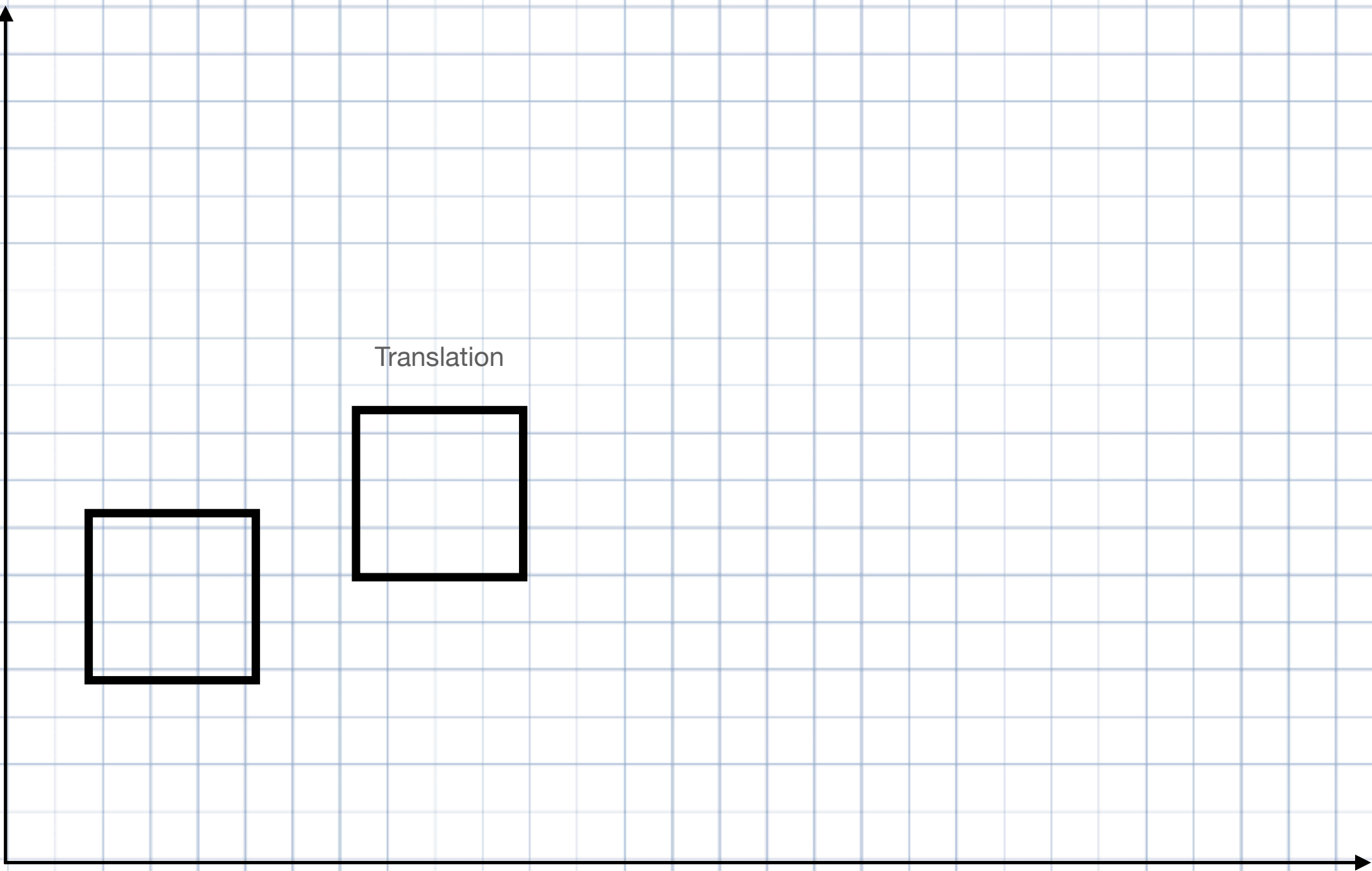
$$\mathbf{x}' = \mathbf{x} + \mathbf{t} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

can be written in homogenous coordinates as

$$\bar{\mathbf{x}}' = \begin{bmatrix} \mathbf{I}_{2 \times 2} & \mathbf{t} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix} \cdot \bar{\mathbf{x}} = \mathbf{A}_t \bar{\mathbf{x}} \quad , \quad \mathbf{A}_t \in \mathbb{R}^{3 \times 3}$$

Note that this holds for any homogenous coordinates:

$$\tilde{\mathbf{x}}' = \begin{bmatrix} \mathbf{I}_{2 \times 2} & \mathbf{t} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix} \cdot \tilde{\mathbf{x}}$$



2D rotation

We can use the 2D rotation matrix

$$\mathbf{R}_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

to rotate a 2D point around the origin by θ via matrix multiplication.

Note: $\mathbf{R}$ is orthonormal, thus

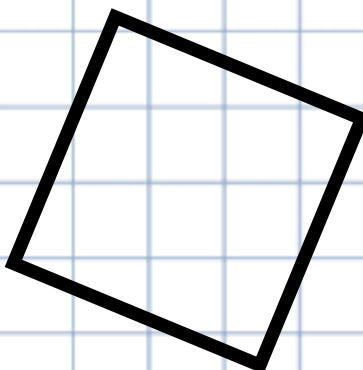
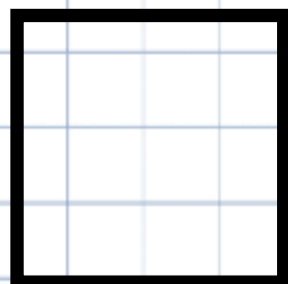
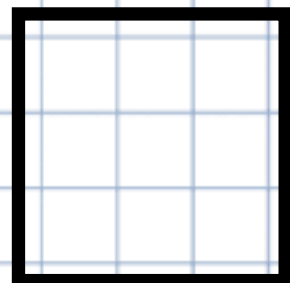
- $\|\mathbf{R}\|^2 = 1$
- $\mathbf{R}\mathbf{R}^\top = \mathbf{I}$

Rigid body transformation

We can perform the joint rotation and translation operation $\mathbf{x}' = \mathbf{R}\mathbf{x} + \mathbf{t}$ using the rigid body transformation matrix

$$\tilde{\mathbf{x}}' = \begin{bmatrix} \mathbf{R}_\theta & \mathbf{t} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix} \tilde{\mathbf{x}}$$

This transformation preserves distances.



Translation

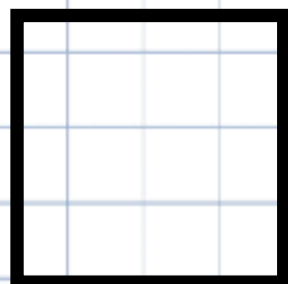
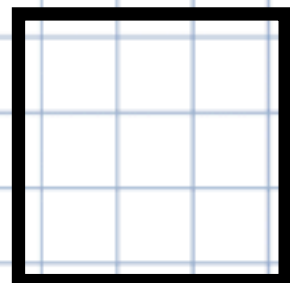
Rigid

Similarity transformation

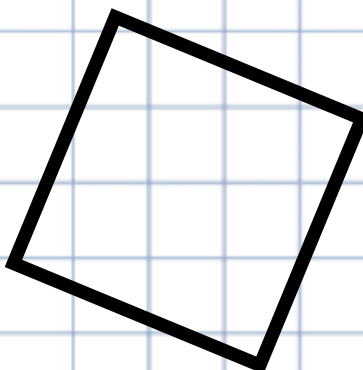
We can allow an additional scaling s to obtain

$$\tilde{\mathbf{x}}' = \begin{bmatrix} s\mathbf{R}_\theta & \mathbf{t} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix} \tilde{\mathbf{x}}$$

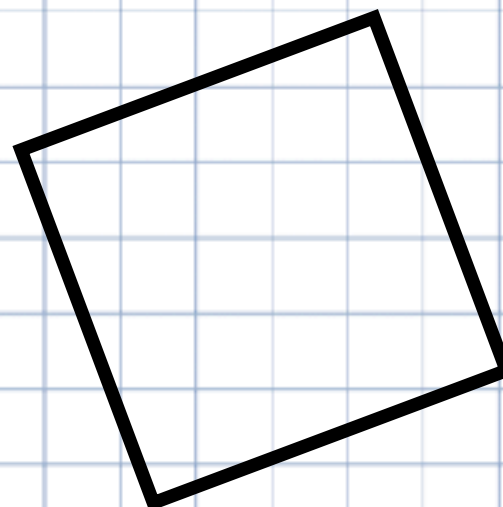
This transformation allows to „resize“ objects, but still preserves angles.



Translation



Rigid



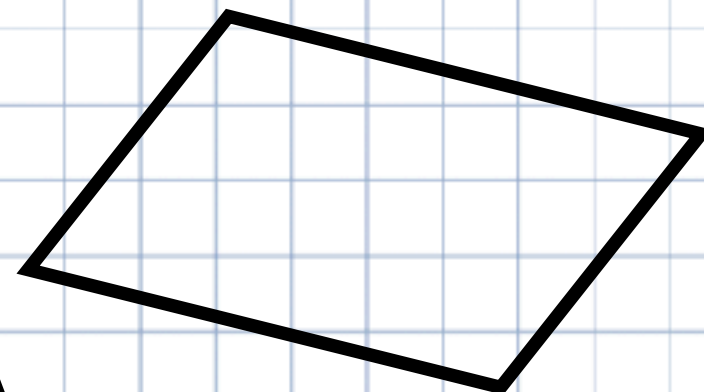
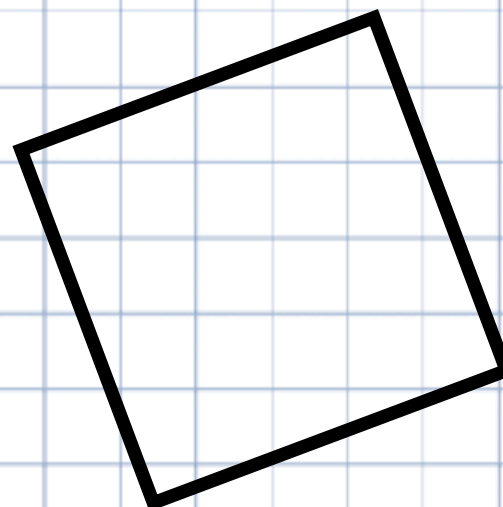
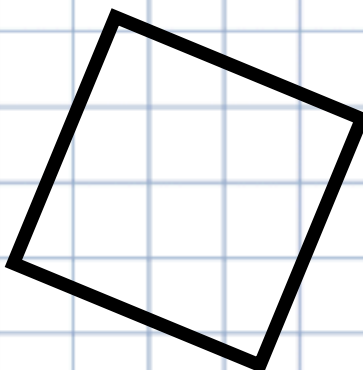
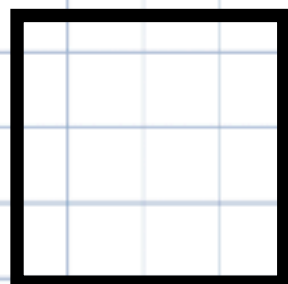
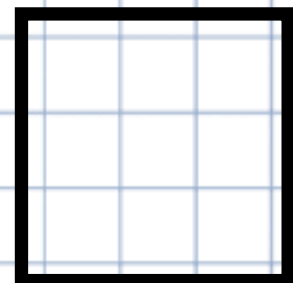
Similarity

Affine transformation

The linear transformation also allows for shearing using unconstrained upper rows

$$\tilde{\mathbf{x}}' = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \tilde{\mathbf{x}}$$

This transformation does not preserve angles any more, but still preserves parallel lines.



Translation

Rigid

Similarity

Affine

Perspective transformation

Using the full 3×3 matrix, we obtain the perspective transformation

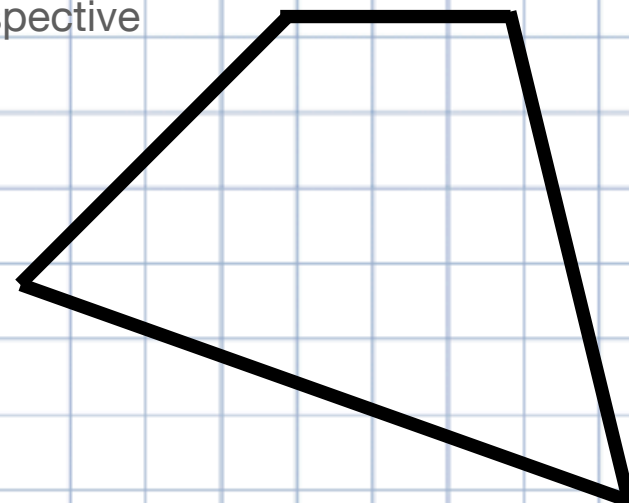
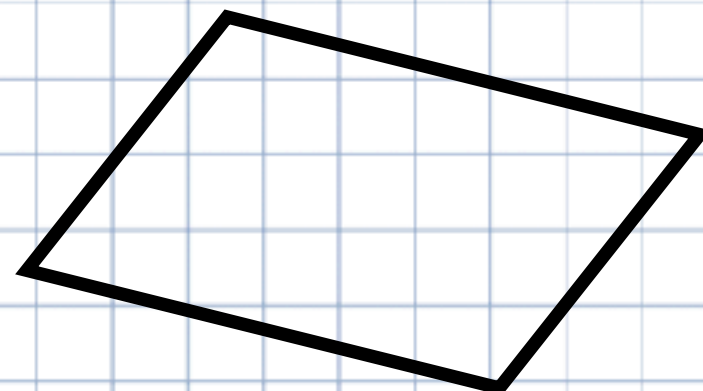
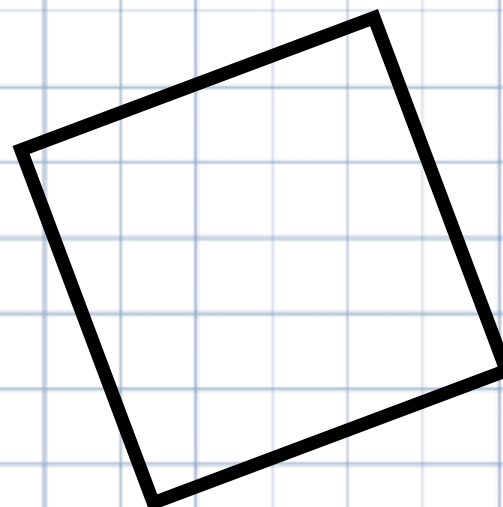
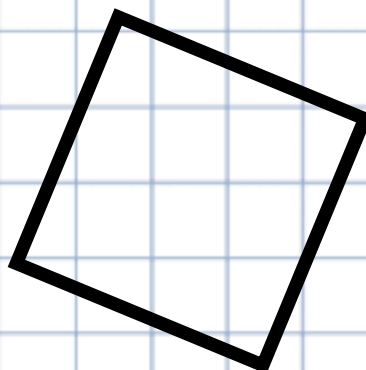
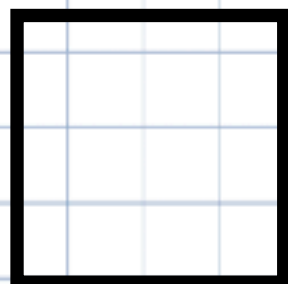
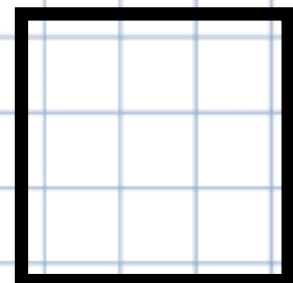
$$\tilde{\mathbf{x}}' = \tilde{\mathbf{H}} \cdot \tilde{\mathbf{x}} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \tilde{\mathbf{x}}$$

$\tilde{\mathbf{H}}$ only preserves straight lines.

It requires homogeneous coordinates.

The Euclidean result requires normalization, thus

$$x' = \frac{h_{11}x + h_{12}y + h_{13}w}{h_{31}x + h_{32}y + h_{33}w} \quad \text{and} \quad y' = \frac{h_{21}x + h_{22}y + h_{23}w}{h_{31}x + h_{32}y + h_{33}w}$$



Translation

Rigid

Similarity

Affine

Perspective